
HOW_TO.pdf - Scenario B

Smart Parking Monitoring

---

1. Prerequisites

Hardware

- Raspberry Pi 3
- MicroSD (≥ 32 GB)
- 2 × LilyGO TTGO LoRa 868 MHz
- DJI Drone (or another drone equipped with a camera)
- Smartphone with Telegram installed
- USB cables
- Wi-Fi connection
- PC with Arduino IDE
- PC running Node-RED

Software

- Raspberry Pi Imager
- Arduino IDE
- Python 3
- Node-RED

---

2. Raspberry Pi Configuration

Install Raspberry Pi OS

1. Install Raspberry Pi Imager from https://www.raspberrypi.com/software/
2. Insert the microSD.
3. Select:

   ➞ **Device** Raspberry Pi 3

   ➞ **OS** Raspberry Pi OS (64-bit)

➞ **Storage** MicroSD

Before writing the image enable:

- SSH
- Wi-Fi
- Username
- Password

Connect via SSH

1. Find the Raspberry IP by writing hostname -I

   Example: 192.168.0.108

2. Connect ssh davide4ne@192.168.0.108 (replace **davide4ne** with your own username)

---

3. Create Python Virtual Environment

1. python3 -m venv venv
2. source venv/bin/activate
3. pip install -r requirements.txt

---

4. Create the Telegram Service

1. Create the service file:

   sudo nano /etc/systemd/system/telegram_bot.service

2. Insert the following configuration (adapt the paths according to your installation):

```
[Unit]
Description=Telegram Bot Service
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/car_project
ExecStart=/home/pi/car_project/venv/bin/python /home/pi/car_project/bot.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Note: Replace pi with your Raspberry Pi username and update both WorkingDirectory and ExecStart to match the location of your project.

3. Reload the systemd configuration:

```
sudo systemctl daemon-reload
```

4. Enable the service at system startup:

```
sudo systemctl enable telegram_bot.service
```

5. Start the service:

```
sudo systemctl start telegram_bot.service
```

6. Verify that the service is running correctly:

```
sudo systemctl status telegram_bot.service
```

If everything has been configured correctly, the service will automatically start every time the Raspberry Pi boots and continuously monitor the Telegram Bot for new images.

5. Telegram Bot Configuration

The Raspberry Pi continuously executes the Python service (bot.py) that performs polling on a dedicated Telegram Bot.

Whenever a new image is received, the following operations are automatically performed:

- image download;
- local storage;
- AI inference;
- vehicle counting;
- LoRa transmission.

To configure the Telegram Bot:

1. Open Telegram.
2. Search **@BotFather**.
3. Create a new Bot that will be the INPUT BOT.
4. Save the generated TOKEN.
5. Create a .env file.
6. Insert TELEGRAM_TOKEN=xxxxxxxxxxxxxxxx

The .env file is intentionally excluded from the repository.

---

6. AI Model

The repository already contains the trained model model_visdrone_yolov8n_car.tflite. No additional training is required. The model has already been trained on the VisDrone dataset and converted into TensorFlow Lite format.

If you want to train another model, you can use the VisDrone dataset following the training procedure described in the project documentation and then convert the final model into TensorFlow Lite.

---

7. LoRa TX Configuration

Several Arduino sketches are provided inside the repository.

Each sketch corresponds to a different experimental campaign.

Final implementations

- LoRa_TX_Car_Arch
- LoRa_TX_Car_Ing

Connectivity tests

- LoRa_TX_Test_Arch
- LoRa_TX_Test_Ing

Select the sketch corresponding to your experiment. The sketches designated "Ing" are characterized by a more robust transmission.

Then:

1. Open Arduino IDE.
2. Load the selected sketch.
3. Select the correct COM port.
4. Select the correct LilyGO TTGO board.
5. Compile.
6. Upload.

Pay attention about the baud rate, it must be the same between the one which is in your code and the one you can see at the bottom right of the page after you click on serial monitor (top right).

The communication parameters defined inside the “Arch” implementation are:

- Frequency: 868 MHz
- SF: 12
- BW: 125 kHz
- CR: 4/8
- Sync Word: 0x12
- CRC Enabled

While inside the “Ing” implementation, the only difference is:

- BW: 62.5 kHz

---

8. LoRa RX Configuration

The receiver sketches follow the same organization.

Final implementations

- LoRa_RX_Car_Arch
- LoRa_RX_Car_Ing

Connectivity tests

- LoRa_RX_Test_Arch
- LoRa_RX_Test_Ing

Select the sketch corresponding to your experiment. The sketches designated "Ing" are characterized by a more robust reception.

Open it in Arduino IDE and upload it to the receiver board, you can follow the steps illustrated in the previous paragraph.

Use exactly the same communication parameters configured on the transmitter.

---

8. Create Output Bot

Create another Telegram Bot using @BotFather. It will be the output bot. Once you have done it, you will receive a TOKEN. Preserve it, we will use it on Node-RED.

---

9. Configure Node-RED

1. Start Node-RED by writing on the terminal of your laptop

   node-red.cmd

2. Before importing the flow, install the required Node-RED packages through

   Menu → Manage Palette → Install

   The Required packages are:

   - node-red-dashboard
   - node-red-node-serialport
   - node-red-node-sqlite
   - node-red-contrib-telegrambot

3. Import flow_car.json
4. Configure
   - **Serial node**
     - verify the serial port connected to the LoRa receiver.
   - **SQLite node**
     - configure the local database path.
   - **MQTT Broker**
     - verify that Mosquitto (or another broker) is correctly installed if MQTT is used.
   - **Telegram sender**
     - you must change the TOKEN with the one from your bot. You must change the chat ID on the previous function node too. To get it:
       - Put a telegram receiver node on the flow (it's not necessary it is linked with something)
       - Put a debug node after it and link the two nodes
       - Deploy and then write something on the chat with your bot
       - You can see the chat ID at the debug window on the right
5. Remove the Telegram receiver and the debug node, then Deploy.

---

10. Drone Image Acquisition

1. Power on the drone.
2. Acquire an image of the parking area.
3. Open Telegram on your smartphone.

4. Send the acquired image to the configured Telegram Bot.
5. The Raspberry Pi will automatically detect the new image and start the complete inference pipeline.

No manual execution of Python scripts is required.

---

11. Running the System

The complete pipeline is managed by bot.py. The script continuously performs Telegram polling. Whenever a new image is received, the following operations are automatically executed:

Receive Image -> Download Image -> AI Inference -> Vehicle Counting -> Generate JSON -> LoRa Transmission

The AI inference is performed by vehicle_detector.py while LoRa communication is handled by lora_send.py

---

11. Communication Protocol

The Generated payload is something similar to

{"id":1718708423,

"count":32}

The Receiver ACK is something similar to

{"ack":1718708423}

The ACK contains the packet identifier used to verify the successful reception of the transmitted message. It is useful in case of retransmission of a packet from the transmitter, in order to know if an information is new or not.

---

12. Verify Correct Execution

The following components should operate correctly:

- Telegram Bot receives the image
- Raspberry Pi downloads the image
- AI inference executes successfully
- Vehicle count is correctly generated

🟩 LoRa packet transmitted

🟩 ACK received

🟩 Node-RED dashboard updated

🟩 SQLite database updated

🟩 Telegram message received from the Output bot

---

13. Troubleshooting

Telegram Bot not receiving images

Verify

- Internet connection
- Bot Token
- Polling service running

AI inference fails

Verify

model_visdrone_yolov8n_car.tflite

exists inside the repository.

LoRa communication fails

Verify

- Frequency
- SF
- BW
- Coding Rate
- Sync Word
- CRC

must match on both devices.

Node-RED dashboard not updating

Verify

- serial connection
- imported flow
- SQLite configuration
- MQTT broker (if used)

---

14. Future Improvements

Possible improvements include:

- Camera installation at a properly height above the parking
- solar-powered Raspberry Pi;
- multi-hop LoRa communication;
- payload optimization;
- adaptive communication parameters.

---

15. Little Tips

The repository contains different Arduino sketches corresponding to different experimental campaigns.

The folders containing **"Test"** implement connectivity validation experiments performed before the final parking monitoring tests.

The folders **"Arch"** and **"Ing"** correspond to the final implementations used during the experiments carried out at the **Architecture Tower** and at the **Engineering Department**, respectively.

When reproducing the project, always choose the Arduino sketches corresponding to the experimental scenario you intend to replicate.